

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Algoritma Sorting (Insertion Sort, Selection Sort, dan Bubble Sort)”

DISUSUN OLEH:

NOFRI ILHAM

2511531013

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

M Galid A



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji 2dalam penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan laporan praktikum Struktur Data dengan materi *Algoritma Sorting* yang membahas *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk dokumentasi hasil praktikum sekaligus untuk memahami konsep dasar algoritma pengurutan data dalam pemrograman. Pada praktikum ini, penulis mempelajari cara kerja beberapa metode sorting, yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*, serta bagaimana proses pengurutan data dilakukan dari data acak menjadi terurut.

Melalui praktikum ini, penulis dapat memahami perbedaan cara kerja masing-masing algoritma sorting, mulai dari proses pertukaran data, pencarian elemen terkecil atau terbesar, hingga proses penyisipan data pada posisi yang sesuai. Selain itu, praktikum ini juga membantu meningkatkan pemahaman penulis mengenai efisiensi algoritma dan penerapannya dalam pemrograman.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang.

Akhir kata, penulis berharap semoga laporan ini dapat memberikan manfaat serta menambah wawasan bagi penulis maupun pembaca.

Wassalamu'alaikum Warahmatullahi Wabarakatuh

Padang, 21 Mei 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR.....	2
DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan.....	4
1.3 Manfaat	5
BAB II PEMBAHASAN	6
2.1 Pengertian Algoritma Sorting.....	6
2.2 Jenis-jenis Algoritma Sorting.....	6
2.2.1 Buble Sort	6
2.2.2 Selection Sort	7
2.2.3 Insertion Sort.....	8
2.3 Implementasi Stack pada Praktikum	9
2.3.1 Kode Pertama(SelectionSort_2511531013).....	9
2.3.2 Kode Kedua(BubleSort_2511531013).....	10
2.3.3 Kode Ketiga(InsertionSort_2511531013).....	12
2.3.4 Kode Keempat(InsertionSortGUI_2511531013)	14
BAB III PENUTUP	19
3.1 Kesimpulan	19
3.2 Saran.....	19
DAFTAR PUSTAKA	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengolahan data yang teratur dan efisien sangat penting agar data lebih mudah digunakan dan diproses. Salah satu proses yang sering dilakukan dalam pengolahan data adalah pengurutan data (*sorting*). Algoritma *sorting* digunakan untuk mengurutkan data berdasarkan urutan tertentu, seperti dari kecil ke besar atau sebaliknya.

Terdapat berbagai macam algoritma *sorting* yang dapat digunakan, di antaranya adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*. Masing-masing algoritma memiliki cara kerja yang berbeda dalam melakukan proses pengurutan data. Dengan mempelajari algoritma *sorting*, mahasiswa dapat memahami bagaimana proses pengurutan dilakukan serta mengetahui kelebihan dan kekurangan dari setiap metode *sorting*.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami konsep dasar algoritma *sorting* dan dapat mengimplementasikannya ke dalam program menggunakan pemrograman Java.

1.2 Tujuan

Adapun tujuan dari praktikum struktur data dengan materi *Algoritma Sorting* adalah sebagai berikut:

1. Memahami konsep dasar algoritma *sorting* dalam pemrograman.
2. Mengetahui cara kerja algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
3. Mampu mengimplementasikan algoritma *sorting* menggunakan pemrograman Java.
4. Melatih kemampuan dalam mengurutkan data menggunakan berbagai metode *sorting*.
5. Memahami perbedaan proses pengurutan pada setiap algoritma *sorting*.

1.3 Manfaat

Adapun manfaat dari praktikum struktur data dengan materi *Algoritma Sorting* adalah sebagai berikut:

1. Mahasiswa dapat memahami proses pengurutan data dalam pemrograman.
2. Menambah pemahaman mengenai cara kerja algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
3. Melatih kemampuan logika pemrograman dalam proses pengurutan data.
4. Membantu mahasiswa memahami penggunaan algoritma sorting sesuai kebutuhan program.
5. Menambah wawasan mengenai pentingnya efisiensi algoritma dalam pengolahan data.

BAB II

PEMBAHASAN

2.1 Pengertian Algoritma Sorting

Algoritma sort adalah metode atau langkah logis untuk mengurutkan sejumlah data. Data yang diurutkan dapat berupa angka, huruf, nama siswa, nilai barang, dan lain-lain.

Pengurutan dilakukan untuk:

1. memudahkan proses pencarian (searching),
2. mempercepat pengolahan data,
3. menghindari duplikasi,
4. meningkatkan performa aplikasi atau database.

2.2 Jenis-jenis Algoritma Sorting

2.2.1 Bubble Sort

Bubble Sort Merupakan algoritma pengurutan paling tua dengan metode pengurutan paling sederhana. Pengurutan yang dilakukan dengan membandingkan masing-masing *item* dalam suatu list secara berpasangan, menukar *item* jika diperlukan, dan mengulaginya sampai akhir list secara berurutan, sehingga tidak ada lagi *item* yang dapat ditukar. Diberi Nama “Bubble” Karena Proses Pengurutan Secara Berangsur-Angsur Bergerak/Berpindah Ke Posisinya Yang Tepat, Seperti Gelembung Yang Keluar Dari Sebuah Gelas Bersoda. Bubble Sort Berhenti Jika Seluruh Array Telah Diperiksa Dan Tidak Ada Pertukaran Lagi Yang Bisa Dilakukan, Serta Tercapai Perurutan Yang Telah Diinginkan.

Kelebihan:

- Metode Bubble Sort Merupakan Metode Yang Paling Sederhana
- Metode Bubble Sort Mudah Dipahami Algoritmanya
- Mudah Untuk Diubah Menjadi Kode.
- Definisi Terurut Terdapat Dengan Jelas Dalam Algoritma.

- Cocok Untuk Pengurutan Data Dengan Elemen Kecil Telah Terurut

Kekurangan:

- Bubble Sort merupakan metode pengurutan yang paling tidak efisien.
- Pada saat mengurutkan data yang sangat besar akan mengalami kelambatan luar biasa, atau dengan kata lain kinerja memburuk cukup signifikan 7 dalam data yang diolah jika data cukup banyak.

2.2.2 Selection Sort

Ide utama dari algoritma *selection sort* adalah memilih elemen dengan nilai paling rendah dan menukar elemen yang terpilih dengan elemen ke- i . Nilai dari i dimulai dari 1 ke n , dimana n adalah jumlah total elemen dikurangi 1.

Kelebihan:

- Algoritma ini sangat rapat dan mudah untuk diimplementasikan.
- Mempercepat pencarian
- Mudah menentukan data maksimum /minimum.
- Mudah menggabungkannya kembali. Kompleksitas selection sort relatif lebih kecil.

Kekurangan:

- Membutuhkan method tambahan
- Sulit untuk digabungkan kembali
- Perlu dihindari untuk penggunaan data lebih dari 1000 tabel, karena akan menyebabkan kompleksitas yang lebih tinggi dan kurang praktis

2.2.3 Insertion Sort

Algoritma *insertion sort* pada dasarnya memilah data yang akan diurutkan menjadi dua bagian, yang belum diurutkan dan yang sudah diurutkan. Elemen pertama diambil dari bagian array yang belum diurutkan dan kemudian diletakkan sesuai posisinya pada bagian lain dari array yang telah diurutkan. Langkah ini dilakukan secara berulang hingga tidak ada lagi elemen yang tersisa pada bagian array yang belum diurutkan.

Kelebihan:

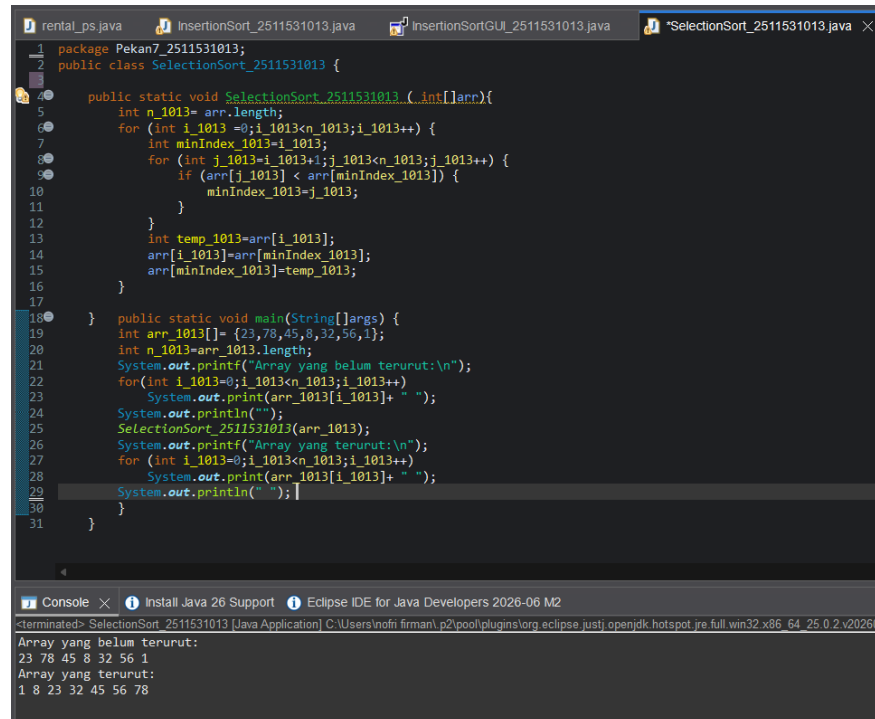
- Sederhana dalam penerapannya.
- Mangkus dalam data yang kecil.
- Jika list sudah terurut atau 8dalam88 terurut maka Insertion Sort akan lebih cepat dibandingkan dengan Quicksort.
- Mangkus dalam data yang 8dalam88 sudah terurut.
- Lebih mangkus 8dalam88g Bubble Sort dan Selection Sort.
- Loop dalam pada Inserion Sort sangat cepat, sehingga membuatnya salah satu algoritma pengurutan tercepat pada jumlah elemen yang sedikit.
- Stabil.

Kekurangan:

- Banyaknya operasi yang diperlukan dalam mencari posisi yang tepat untuk elemen larik.
- Untuk larik yang jumlahnya besar ini tidak praktis.
- Jika list terurut terbalik sehingga setiap eksekusi dari perintah harus memindai dan mengganti seluruh bagian sebelum menyisipkan elemen berikutnya.
- Membutuhkan waktu $O(n^2)$ pada data yang tidak terurut, sehingga tidak cocok dalam pengurutan elemen dalam jumlah besar.

2.3 Impementasi Stack pada Praktikum

2.3.1 Kode Pertama(SelectionSort_2511531013)



```
1 package Pekan7_2511531013;
2 public class SelectionSort_2511531013 {
3
4     public static void SelectionSort_2511531013 (int[] arr){
5         int n_1013= arr.length;
6         for (int i_1013 =0; i_1013<n_1013; i_1013++) {
7             int minIndex_1013=i_1013;
8             for (int j_1013=i_1013+1; j_1013<n_1013; j_1013++) {
9                 if (arr[j_1013] < arr[minIndex_1013]) {
10                     minIndex_1013=j_1013;
11                 }
12             }
13             int temp_1013=arr[i_1013];
14             arr[i_1013]=arr[minIndex_1013];
15             arr[minIndex_1013]=temp_1013;
16         }
17     }
18     public static void main(String[] args) {
19         int arr_1013[] = {23,78,45,8,32,56,1};
20         int n_1013=arr_1013.length;
21         System.out.printf("Array yang belum terurut:\n");
22         for(int i_1013=0; i_1013<n_1013; i_1013++)
23             System.out.print(arr_1013[i_1013]+ " ");
24         System.out.println("");
25         SelectionSort_2511531013(arr_1013);
26         System.out.printf("Array yang terurut:\n");
27         for (int i_1013=0; i_1013<n_1013; i_1013++)
28             System.out.print(arr_1013[i_1013]+ " ");
29         System.out.println("");
30     }
31 }
```

Console

```
<terminated> SelectionSort_2511531013 [Java Application] C:\Users\inofirman\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_25.0.2.v20260
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78
```

Kode 2.1

Program SelectionSort_2511531013 digunakan untuk mengurutkan data menggunakan algoritma *Selection Sort*. Pada method SelectionSort_2511531013(int[] arr), program menerima parameter berupa array integer yang akan diurutkan. Variabel n_1013 digunakan untuk menyimpan 9 dalam 9 array.

Perulangan pertama menggunakan 9 dalam 9 i_1013 yang berfungsi untuk menentukan posisi data yang akan diisi dengan nilai terkecil. Pada setiap perulangan, 9 dalam 9 minIndex_1013 diisi dengan indeks awal yang sedang dicek.

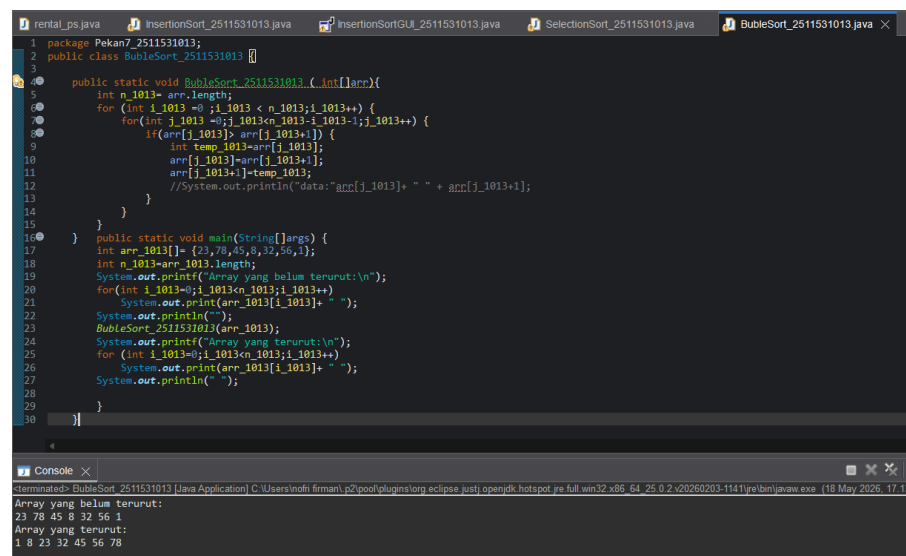
Selanjutnya, perulangan kedua menggunakan 9 dalam 9 j_1013 untuk mencari nilai terkecil pada bagian array yang belum

terurut. Jika ditemukan nilai yang lebih kecil dari `arr[minIndex_1013]`, maka indeks terkecil akan diperbarui ke posisi tersebut.

Setelah nilai terkecil ditemukan, program melakukan proses pertukaran data (*swap*). Nilai pada posisi `i_1013` disimpan sementara ke `temp_1013`, kemudian nilai terkecil dipindahkan ke posisi depan, dan nilai lama dipindahkan ke posisi sebelumnya.

Pada method `main`, dibuat array `arr_1013` dengan isi data 23, 78, 45, 8, 32, 56, 1. Program terlebih dahulu menampilkan array sebelum diurutkan. Setelah itu, method `SelectionSort_2511531013()` dipanggil untuk mengurutkan data dari kecil ke besar.

2.3.2 Kode Kedua(BubleSort_2511531013)



```
1 package Pekan7_2511531013;
2 public class BubleSort_2511531013 {
3
4     public static void BubleSort_2511531013(int[] arr){
5         int n_1013= arr.length;
6         for (int i_1013=0; i_1013 < n_1013; i_1013++){
7             for (int j_1013=0; j_1013 < n_1013-i_1013; j_1013++){
8                 if(arr[j_1013]> arr[j_1013+1]){
9                     int temp_1013=arr[j_1013];
10                    arr[j_1013]=arr[j_1013+1];
11                    arr[j_1013+1]=temp_1013;
12                    //System.out.println("data: "+arr[j_1013]+ " " + arr[j_1013+1]);
13                }
14            }
15        }
16    }
17    public static void main(String[] args) {
18        int arr_1013[]={23,78,45,8,32,56,1};
19        int n_1013=arr_1013.length;
20        System.out.println("Array yang belum terurut:\n");
21        for (int i_1013=0; i_1013 < n_1013; i_1013++){
22            System.out.print(arr_1013[i_1013]+ " ");
23        }
24        System.out.println("\n");
25        BubleSort_2511531013(arr_1013);
26        System.out.println("Array yang terurut:\n");
27        for (int i_1013=0; i_1013 < n_1013; i_1013++){
28            System.out.print(arr_1013[i_1013]+ " ");
29        }
30        System.out.println("\n");
31    }
32 }
```

Console

```
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78
```

Kode2.2

Program `BubbleSort_2511531013` digunakan untuk mengurutkan data menggunakan algoritma *Bubble Sort*. Pada method `BubbleSort_2511531013(int[] arr)`, program menerima parameter berupa array integer yang akan diurutkan. Variabel `n_1013` digunakan untuk menyimpan `11` dalam `11` array.

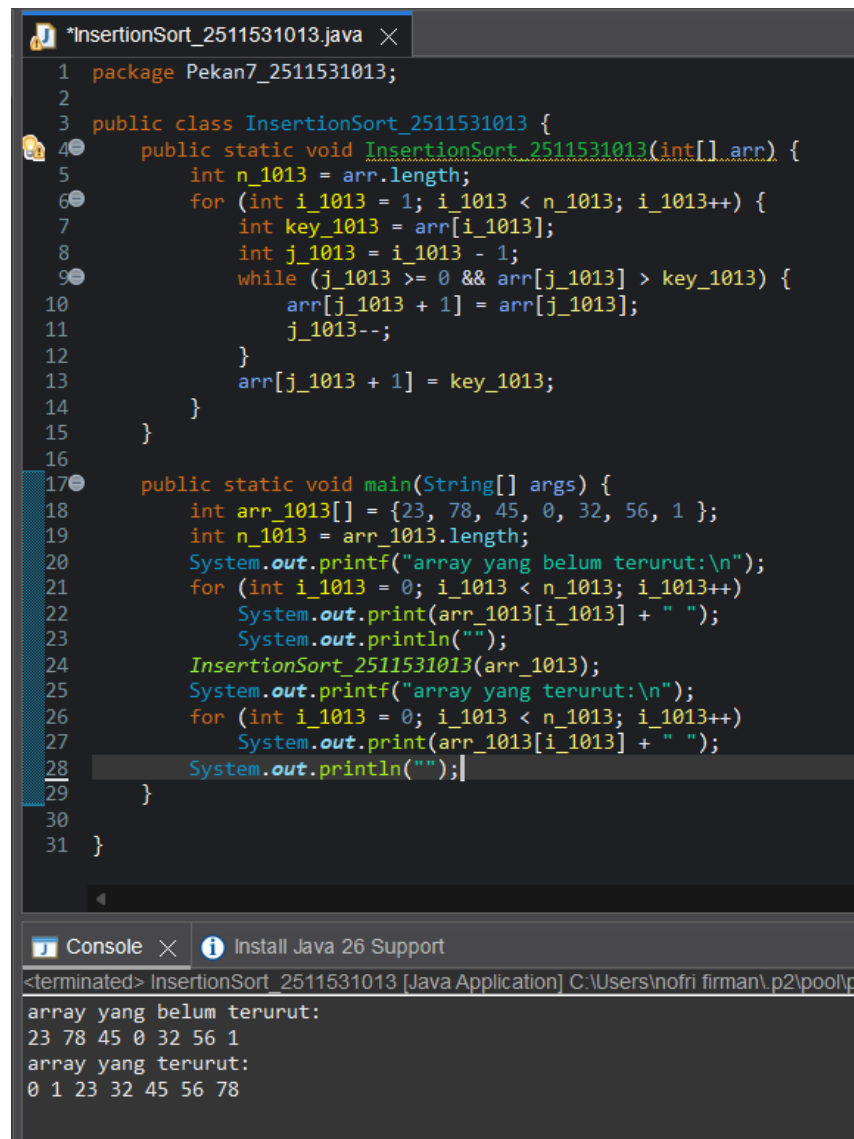
Perulangan pertama menggunakan `11` dalam `1111 i_1013` yang berfungsi untuk menghitung jumlah tahap pengurutan. Pada setiap tahap, data terbesar akan berpindah ke posisi paling akhir.

Perulangan kedua menggunakan `11` dalam `1111 j_1013` untuk membandingkan dua elemen yang berdekatan, yaitu `arr[j_1013]` dan `arr[j_1013+1]`. Jika nilai di sebelah kiri lebih besar dari nilai di sebelah kanan, maka kedua data akan ditukar menggunakan `11` dalam `1111` sementara `temp_1013`.

Proses pertukaran data dilakukan dengan menyimpan nilai sementara ke `temp_1013`, kemudian nilai kanan dipindahkan ke kiri, dan nilai sementara dipindahkan ke kanan. Dengan proses ini, data akan bergerak secara bertahap sampai tersusun dari kecil ke besar.

Pada method `main`, dibuat array `arr_1013` dengan isi data 23, 78, 45, 8, 32, 56, 1. Program terlebih dahulu menampilkan array sebelum diurutkan. Setelah itu, method `BubbleSort_2511531013()` dipanggil untuk melakukan proses pengurutan.

2.3.3 Kode Ketiga(InsertionSort_2511531013)



```
1 package Pekan7_2511531013;
2
3 public class InsertionSort_2511531013 {
4     public static void InsertionSort_2511531013(int[] arr) {
5         int n_1013 = arr.length;
6         for (int i_1013 = 1; i_1013 < n_1013; i_1013++) {
7             int key_1013 = arr[i_1013];
8             int j_1013 = i_1013 - 1;
9             while (j_1013 >= 0 && arr[j_1013] > key_1013) {
10                 arr[j_1013 + 1] = arr[j_1013];
11                 j_1013--;
12             }
13             arr[j_1013 + 1] = key_1013;
14         }
15     }
16
17     public static void main(String[] args) {
18         int arr_1013[] = {23, 78, 45, 0, 32, 56, 1};
19         int n_1013 = arr_1013.length;
20         System.out.printf("array yang belum terurut:\n");
21         for (int i_1013 = 0; i_1013 < n_1013; i_1013++)
22             System.out.print(arr_1013[i_1013] + " ");
23         System.out.println("");
24         InsertionSort_2511531013(arr_1013);
25         System.out.printf("array yang terurut:\n");
26         for (int i_1013 = 0; i_1013 < n_1013; i_1013++)
27             System.out.print(arr_1013[i_1013] + " ");
28         System.out.println("");
29     }
30 }
31 }
```

Console

```
<terminated> InsertionSort_2511531013 [Java Application] C:\Users\nofri firman\p2\pool\p
array yang belum terurut:
23 78 45 0 32 56 1
array yang terurut:
0 1 23 32 45 56 78
```

Kode 2.3

Program InsertionSort_2511531013 digunakan untuk mengurutkan data menggunakan algoritma *Insertion Sort*. Pada method InsertionSort_2511531013(int[] arr), program menerima parameter berupa array integer yang akan diurutkan. Variabel n_1013 digunakan untuk menyimpan 12dalam12 array.

Perulangan menggunakan 12dalam12 i_1013 dimulai dari indeks 1 karena elemen pertama dianggap sudah terurut. Pada setiap

perulangan, nilai array pada posisi `i_1013` disimpan ke dalam `key_1013`. Variabel ini digunakan sebagai data yang akan disisipkan ke posisi yang tepat.

Selanjutnya, `j_1013` digunakan untuk membandingkan data sebelumnya dengan `key_1013`. Selama data sebelumnya lebih besar dari `key_1013`, maka data tersebut akan digeser satu posisi ke kanan. Proses ini dilakukan menggunakan perulangan `while`.

Setelah posisi yang sesuai ditemukan, nilai `key_1013` akan ditempatkan pada posisi yang tepat, sehingga bagian array dari kiri sampai posisi tertentu tetap dalam keadaan terurut.

Pada method `main`, dibuat array `arr_1013` dengan isi data 23, 78, 45, 0, 32, 56, 1. Program terlebih dahulu menampilkan array sebelum diurutkan. Setelah itu, method `InsertionSort_2511531013()` dipanggil untuk melakukan proses pengurutan.

2.3.4 Kode Keempat(InsertionSortGUI_2511531013)

```
InsertionSort_2511531013.java InsertionSortGUI_2511531013.java SelectionSort_2511531013.java
1 package Pekan7_2511531013;
2
3 import java.awt.*;
4 import javax.swing.*;
5
6 public class InsertionSortGUI_2511531013 extends JFrame {
7     private static final long serialVersionUID = 1L;
8     private int[] array_1013;
9     private JLabel [] labelArray_1013;
10    private JButton stepButton_1013, resetButton_1013, setButton_1013;
11    private JTextField inputField_1013;
12    private JPanel panelArray_1013;
13    private JTextArea stepArea_1013;
14
15    private int i_1013 = 1 ,j_1013;
16    private boolean sorting_1013 = false;
17    private int stepCount_1013 = 1;
18
19    public InsertionSortGUI_2511531013() {
20        setTitle("Insertion Sort Langkah per Langkah");
21        setSize(750, 400);
22        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
23        setLocationRelativeTo(null);
24        setLayout(new BorderLayout());
25
26        // Panel input
27        JPanel inputPanel_1013 = new JPanel(new FlowLayout());
28        inputField_1013 = new JTextField(30);
29        setButton_1013 = new JButton("Set Array");
30        inputPanel_1013.add(new JLabel("Masukkan angka (pisahkan dengan koma): "));
31        inputPanel_1013.add(inputField_1013);
32        inputPanel_1013.add(setButton_1013);
33
34        // Panel array visual
35        panelArray_1013 = new JPanel();
36        panelArray_1013.setLayout(new FlowLayout());
37
38        // Panel kontrol
39        JPanel controlPanel_1013 = new JPanel();
40        stepButton_1013 = new JButton("Langkah Selanjutnya");
41        resetButton_1013 = new JButton("Reset");
42        stepButton_1013.setEnabled(false);
43        controlPanel_1013.add(stepButton_1013);
44        controlPanel_1013.add(resetButton_1013);
45    }
```

```

46 // Area teks untuk log langkah-langkah
47 stepArea_1013 = new JTextArea(8, 60);
48 stepArea_1013.setEditable(false);
49 stepArea_1013.setFont(new Font("Monospaced", Font.PLAIN, 14));
50 JScrollPane scrollPane_1013 = new JScrollPane(stepArea_1013);
51
52 // Tambahkan panel ke frame
53 add(inputPanel_1013, BorderLayout.NORTH);
54 add(panelArray_1013, BorderLayout.CENTER);
55 add(controlPanel_1013, BorderLayout.SOUTH);
56 add(scrollPane_1013, BorderLayout.EAST);
57
58 // Event Set Array
59 setButton_1013.addActionListener(e -> setArrayFromInput());
60
61 // Event Langkah Selanjutnya
62 stepButton_1013.addActionListener(e -> performStep());
63
64 // Event Reset
65 resetButton_1013.addActionListener(e -> reset());
66
67 }
68 private void setArrayFromInput() {
69     String text_1013 = inputField_1013.getText().trim();
70     if (text_1013.isEmpty()) return;
71     String[] parts_1013 = text_1013.split(",");
72     array_1013 = new int[parts_1013.length];
73     try {
74         for (int k_1013 = 0; k_1013 < parts_1013.length; k_1013++) {
75             array_1013[k_1013] = Integer.parseInt(parts_1013[k_1013].trim());
76         }
77     } catch (NumberFormatException e) {
78         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan "
79             + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
80         return;
81     }
82
83     i_1013 = 1;
84     stepCount_1013 = 1;
85     sorting_1013 = true;
86     stepButton_1013.setEnabled(true);
87     stepArea_1013.setText("");
88     panelArray_1013.removeAll();
89
90     labelArray_1013 = new JLabel[array_1013.length];

```

```

91     for (int k_1013 = 0; k_1013 < array_1013.length; k_1013++) {
92         labelArray_1013[k_1013] = new JLabel(String.valueOf(array_1013[k_1013]));
93         labelArray_1013[k_1013].setFont(new Font("Arial", Font.BOLD, 24));
94         labelArray_1013[k_1013].setBorder(BorderFactory.createLineBorder(Color.BLACK));
95         labelArray_1013[k_1013].setPreferredSize(new Dimension(50, 50));
96         labelArray_1013[k_1013].setHorizontalAlignment(SwingConstants.CENTER);
97         panelArray_1013.add(labelArray_1013[k_1013]);
98     }
99
100     panelArray_1013.revalidate();
101     panelArray_1013.repaint();
102 }
103
104 private void performStep() {
105     if (i_1013 < array_1013.length && sorting_1013) {
106         int key_1013 = array_1013[i_1013];
107         j_1013 = i_1013 - 1;
108
109         StringBuilder stepLog = new StringBuilder();
110         stepLog.append("Langkah").append(stepCount_1013).
111             append(": Masukkan ").append(key_1013).append("\n");
112
113         while (j_1013 >= 0 && array_1013[j_1013] > key_1013) {
114             array_1013[j_1013 + 1] = array_1013[j_1013];
115             j_1013--;
116         }
117         array_1013[j_1013 + 1] = key_1013;
118
119         updateLabels();
120         stepLog.append("Hasil: ").append(arrayToString(array_1013)).append("\n\n");
121         stepArea_1013.append(stepLog.toString());
122
123         i_1013++;
124         stepCount_1013++;
125
126         if (i_1013 == array_1013.length) {
127             sorting_1013 = false;
128             stepButton_1013.setEnabled(false);
129             JOptionPane.showMessageDialog(this, "Sorting selesai!");
130         }
131     }
132 }
133 private void updateLabels() {
134     for (int k_1013 = 0; k_1013 < array_1013.length; k_1013++) {
135         labelArray_1013[k_1013].setText(String.valueOf(array_1013[k_1013]));
136     }

```

```

136     }
137 }
138
139 private void reset() {
140     inputField_1013.setText("");
141     panelArray_1013.removeAll();
142     panelArray_1013.revalidate();
143     panelArray_1013.repaint();
144     stepArea_1013.setText("");
145     stepButton_1013.setEnabled(false);
146     sorting_1013 = false;
147     i_1013 = 1;
148     stepCount_1013 = 1;
149 }
150
151 private String arrayToString(int[] arr) {
152     StringBuilder sb = new StringBuilder();
153     for (int k = 0; k < arr.length; k++) {
154         sb.append(arr[k]);
155         if (k < arr.length - 1) sb.append(", ");
156     }
157     return sb.toString();
158 }
159
160 public static void main(String[] args){
161     SwingUtilities.invokeLater(()-> {
162         InsertionSortGUI_2511531013 gui = new InsertionSortGUI_2511531013();
163         gui.setVisible(true);
164     });
165 }
166 }

```

Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

6	7	10
45	54	77
78	894	

Langkah4: Memasukkan 45
Hasil: 6, 7, 10, 45, 894, 78, 54, 77

Langkah5: Memasukkan 78
Hasil: 6, 7, 10, 45, 78, 894, 54, 77

Langkah6: Memasukkan 54
Hasil: 6, 7, 10, 45, 54, 78, 894, 77

Langkah7: Memasukkan 77
Hasil: 6, 7, 10, 45, 54, 77, 78, 894

Insertion Sort Langkah per Langkah

Masukkan angka (pisahkan dengan koma):

6	7	10
45	54	77
78	894	

Langkah1: Memasukkan 6
Hasil: 6, 10, 7, 894, 45, 78, 54, 77

Langkah2: Memasukkan 7
Hasil: 6, 7, 10, 894, 45, 78, 54, 77

Langkah3: Memasukkan 894
Hasil: 6, 7, 10, 894, 45, 78, 54, 77

Langkah4: Memasukkan 45
Hasil: 6, 7, 10, 45, 894, 78, 54, 77

Langkah5: Memasukkan 78
Hasil: 6, 7, 10, 45, 78, 894, 54, 77

Kode 2.4

Program `InsertionSortGUI_2511531013` merupakan implementasi algoritma *Insertion Sort* berbasis GUI (*Graphical User Interface*) menggunakan Java Swing. Program ini digunakan untuk menampilkan proses pengurutan data secara 17dalam17 demi 17dalam17 sehingga pengguna dapat melihat proses sorting dengan lebih jelas.

Program menggunakan class `Jframe` sebagai tampilan utama aplikasi. Pada class ini terdapat beberapa komponen GUI seperti `JtextField`, `Jbutton`, `Jpanel`, `Jlabel`, dan `JtextArea` yang digunakan untuk menerima input, menampilkan array, tombol 17dalam17, serta menampilkan 17dalam17 proses sorting.

Variabel `array_1013` digunakan untuk menyimpan data array yang akan diurutkan. Variabel `labelArray_1013` digunakan untuk menampilkan isi array dalam bentuk visual menggunakan label. Tombol `stepButton_1013` digunakan untuk menjalankan proses sorting 17dalam17 demi 17dalam17, `resetButton_1013` digunakan untuk mengembalikan program ke kondisi awal, sedangkan `setButton_1013` digunakan untuk memasukkan array dari input pengguna.

Constructor `InsertionSortGUI_2511531013()` digunakan untuk mengatur tampilan GUI. Pada bagian ini program mengatur ukuran frame, layout, panel input, panel array, panel 17dalam17, serta area teks untuk menampilkan 17dalam17-langkah sorting. Selain itu, pada constructor juga ditambahkan event listener untuk tombol Set Array, Langkah Selanjutnya, dan Reset.

Method `setArrayFromInput()` digunakan untuk mengambil input array dari pengguna. Data dimasukkan dalam bentuk angka yang dipisahkan dengan koma, kemudian diproses menggunakan `split(",")` dan diubah menjadi tipe integer menggunakan

Integer.parseInt(). Setelah array berhasil dibuat, program akan menampilkan data array dalam bentuk label pada panel visual. Jika input tidak valid, program akan menampilkan pesan error menggunakan JOptionPane.

Method performStep() digunakan untuk menjalankan proses *Insertion Sort* secara bertahap. Pada setiap 18dalam18, program mengambil satu data sebagai key_1013, kemudian membandingkannya dengan data sebelumnya. Jika data sebelumnya lebih besar, maka data akan digeser ke kanan sampai ditemukan posisi yang sesuai untuk key_1013. Setelah proses selesai, hasil array terbaru akan ditampilkan pada GUI dan 18dalam18 sorting dicatat pada stepArea_1013.

Method updateLabels() digunakan untuk memperbarui tampilan label array setelah proses sorting dilakukan. Dengan method ini, perubahan posisi data dapat langsung terlihat pada GUI.

Method reset() digunakan untuk menghapus seluruh input dan tampilan array sehingga program kembali ke kondisi awal. Method ini juga menonaktifkan tombol 18dalam18 dan menghapus log proses sorting.

Method arrayToString() digunakan untuk mengubah isi array menjadi bentuk string agar lebih mudah ditampilkan pada area log langkah sorting.

Pada method main, program dijalankan menggunakan SwingUtilities.invokeLater() agar GUI berjalan pada thread Swing. Setelah itu objek InsertionSortGUI_2511531013 dibuat dan ditampilkan menggunakan setVisible(true).

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting digunakan untuk mengurutkan data agar lebih terstruktur dan mudah diproses. Pada praktikum ini digunakan tiga algoritma sorting, yaitu *Selection Sort*, *Bubble Sort*, dan *Insertion Sort*.

Algoritma *Selection Sort* bekerja dengan mencari nilai terkecil kemudian menukarnya ke posisi yang sesuai. Algoritma *Bubble Sort* bekerja dengan membandingkan dua data yang berdekatan lalu menukarnya jika urutannya salah. Sedangkan algoritma *Insertion Sort* bekerja dengan menyisipkan data ke posisi yang tepat pada bagian array yang sudah terurut.

Melalui praktikum ini, penulis dapat memahami cara kerja masing-masing algoritma sorting serta mengetahui perbedaan proses pengurutannya. Selain itu, penulis juga dapat mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java, baik berbasis console maupun GUI. Praktikum ini juga membantu meningkatkan pemahaman mengenai logika pemrograman dan proses pengolahan data dalam struktur data.

3.2 Saran

Saran untuk praktikum selanjutnya adalah mahasiswa diharapkan lebih banyak berlatih dalam membuat dan memahami algoritma sorting agar lebih memahami proses pengurutan data. Selain itu, mahasiswa juga disarankan mencoba algoritma sorting lainnya seperti *Quick Sort* dan *Merge Sort* untuk menambah wawasan mengenai metode pengurutan data yang lebih efisien.

Dalam penulisan program, mahasiswa juga perlu lebih teliti dalam memahami alur algoritma dan penggunaan syntax agar program dapat berjalan dengan baik tanpa error. Selain itu, penggunaan GUI pada program sorting dapat lebih dikembangkan agar tampilan program menjadi lebih interaktif dan mudah dipahami.

DAFTAR PUSTAKA

- [1] Terapan TI Vokasi Unesa, “Algoritma Sort: Pengertian, Jenis, Cara Kerja dan Contoh Lengkap,” [Daring]. Tersedia pada: <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap> [Diakses: 21-Mei-2026].
- [2] BINUS University, “Macam-Macam Sorting pada Java,” [Daring]. Tersedia pada: <https://sis.binus.ac.id/2024/04/05/macam-macam-sorting-pada-java/> [Diakses: 21-Mei-2026].

LAPORAN TUGAS PEKAN 5

STRUKTUR DATA

DISUSUN OLEH:

NOFRI ILHAM

2511531013

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

1. Kelas ADT_Mahasiswa_2511531013

```
1 package Pekan7_2511531013;
2
3 public class Mahasiswa_2511531013 {
4     private String nama_1013;
5     private String nim_1013;
6     private String prodi_1013;
7
8     public Mahasiswa_2511531013(String nama_1013, String nim_1013, String prodi_1013) {
9         this.nama_1013 = nama_1013;
10        this.nim_1013 = nim_1013;
11        this.prodi_1013 = prodi_1013;
12    }
13
14    public String getNama_1013() {
15        return nama_1013;
16    }
17
18    public String getNim_1013() {
19        return nim_1013;
20    }
21
22    public String getProdi_1013() {
23        return prodi_1013;
24    }
25
26    public void setNama_1013(String nama_1013) {
27        this.nama_1013 = nama_1013;
28    }
29
30    public void setNim_1013(String nim_1013) {
31        this.nim_1013 = nim_1013;
32    }
33
34    public void setProdi_1013(String prodi_1013) {
35        this.prodi_1013 = prodi_1013;
36    }
37
38    @Override
39    public String toString() {
40        return nama_1013 + " - " + nim_1013 + " - " + prodi_1013;
41    }
42 }
```

Program Mahasiswa_2511531013 digunakan sebagai ADT (Abstract Data Type) untuk menyimpan data mahasiswa. Pada class ini terdapat atribut nama_1013, nim_1013, dan prodi_1013 yang digunakan untuk menyimpan nama mahasiswa, NIM mahasiswa, dan program studi mahasiswa.

Program juga memiliki constructor Mahasiswa_2511531013(String nama_1013, String nim_1013, String prodi_1013) yang digunakan untuk membuat objek mahasiswa baru. Constructor ini menerima parameter berupa nama, NIM, dan program studi yang kemudian disimpan ke dalam atribut objek.

Method getter digunakan untuk mengambil nilai atribut seperti nama, NIM, dan program studi. Sedangkan method setter digunakan untuk mengubah data atribut mahasiswa.

2. Kelas GUI_Utama_2511531013

```
1 package Pekan7_2511531013;
2
3 import java.awt.*;
4 import java.util.ArrayList;
5 import javax.swing.*;
6 import javax.swing.table.DefaultTableModel;
7
8 public class GUI_Utama_2511531013 extends JFrame {
9     private static final long serialVersionUID = 1L;
10
11     private ArrayList<Mahasiswa_2511531013> dataMahasiswa_1013 = new ArrayList<>();
12
13     private JTextField namaField_1013, nimField_1013, prodiField_1013;
14     private JButton tambahButton_1013, hapusButton_1013, sortingButton_1013;
15     private JComboBox<String> comboSorting_1013;
16     private JTable tabelMahasiswa_1013;
17     private DefaultTableModel modelTabel_1013;
18     private JTextArea prosesArea_1013;
19
20     public GUI_Utama_2511531013() {
21         setTitle("Sorting Nama Mahasiswa - 2511531013");
22         setSize(850, 550);
23         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24         setLocationRelativeTo(null);
25         getContentPane().setLayout(new BorderLayout());
26
27         // panel input data mahasiswa
28         JPanel inputPanel_1013 = new JPanel(new GridLayout(4, 2, 5, 5));
29         inputPanel_1013.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));
30
31         namaField_1013 = new JTextField();
32         nimField_1013 = new JTextField();
33         prodiField_1013 = new JTextField();
34
35         inputPanel_1013.add(new JLabel("Nama Mahasiswa"));
36         inputPanel_1013.add(namaField_1013);
37
38         inputPanel_1013.add(new JLabel("NIM Mahasiswa"));
39         inputPanel_1013.add(nimField_1013);
40
41         inputPanel_1013.add(new JLabel("Program Studi"));
42         inputPanel_1013.add(prodiField_1013);
43
44         tambahButton_1013 = new JButton("Tambah");
45         hapusButton_1013 = new JButton("Hapus");
46         sortingButton_1013 = new JButton("Sorting");
47
48         inputPanel_1013.add(tambahButton_1013);
49         inputPanel_1013.add(hapusButton_1013);
50         inputPanel_1013.add(sortingButton_1013);
51
52         tabelMahasiswa_1013 = new JTable();
53         modelTabel_1013 = new DefaultTableModel();
54         tabelMahasiswa_1013.setModel(modelTabel_1013);
55
56         prosesArea_1013 = new JTextArea();
57
58         JScrollPane scrollTabel = new JScrollPane(tabelMahasiswa_1013);
59         JScrollPane scrollProses = new JScrollPane(prosesArea_1013);
60
61         getContentPane().add(inputPanel_1013, BorderLayout.NORTH);
62         getContentPane().add(scrollTabel, BorderLayout.CENTER);
63         getContentPane().add(scrollProses, BorderLayout.SOUTH);
64     }
65 }
```

```

44     tambahButton_1013 = new JButton("Tambah Data");
45     hapusButton_1013 = new JButton("Hapus Data");
46
47     inputPanel_1013.add(tambahButton_1013);
48     inputPanel_1013.add(hapusButton_1013);
49
50     // tabel untuk menampilkan data mahasiswa
51     modelTabel_1013 = new DefaultTableModel(new String[]{"Nama", "NIM", "Prodi"}, 0);
52
53     // panel sorting
54     JPanel sortingPanel_1013 = new JPanel(new FlowLayout());
55     comboSorting_1013 = new JComboBox<>{
56         new String[]{"Insertion Sort", "Selection Sort", "Bubble Sort"}
57     };
58     sortingButton_1013 = new JButton("Mulai Sorting");
59
60     sortingPanel_1013.add(new JLabel("Pilih Algoritma Sorting: "));
61     sortingPanel_1013.add(comboSorting_1013);
62     sortingPanel_1013.add(sortingButton_1013);
63
64     // area untuk menampilkan langkah sorting
65     prosesArea_1013 = new JTextArea(12, 30);
66     prosesArea_1013.setEditable(false);
67     prosesArea_1013.setFont(new Font("Monospaced", Font.PLAIN, 13));
68     JScrollPane scrollProses_1013 = new JScrollPane(prosesArea_1013);
69     scrollProses_1013.setBorder(BorderFactory.createTitledBorder("Proses Sorting"));
70
71     getContentPane().add(inputPanel_1013, BorderLayout.NORTH);
72     getContentPane().add(sortingPanel_1013, BorderLayout.SOUTH);
73     getContentPane().add(scrollProses_1013, BorderLayout.EAST);
74     tabelMahasiswa_1013 = new JTable(modelTabel_1013);
75     JScrollPane scrollTabel_1013 = new JScrollPane(tabelMahasiswa_1013);
76     getContentPane().add(scrollTabel_1013, BorderLayout.CENTER);
77
78     tambahButton_1013.addActionListener(e -> tambahData_1013());
79     hapusButton_1013.addActionListener(e -> hapusData_1013());
80     sortingButton_1013.addActionListener(e -> mulaiSorting_1013());
81 }
82
83 // method untuk menambahkan data mahasiswa
84 private void tambahData_1013() {
85     String nama_1013 = namaField_1013.getText().trim();
86     String nim_1013 = nimField_1013.getText().trim();

```



```

87         String prodi_1013 = prodiField_1013.getText().trim();
88
89         if (nama_1013.isEmpty() || nim_1013.isEmpty() || prodi_1013.isEmpty()) {
90             JOptionPane.showMessageDialog(this, "Semua data harus diisi!");
91             return;
92         }
93
94         Mahasiswa_2511531013 mhs_1013 =
95             new Mahasiswa_2511531013(nama_1013, nim_1013, prodi_1013);
96
97         dataMahasiswa_1013.add(mhs_1013);
98         tampilkanData_1013();
99
100        namaField_1013.setText("");
101        nimField_1013.setText("");
102        prodiField_1013.setText("");
103    }
104
105    // method untuk menghapus data berdasarkan baris yang dipilih
106    private void hapusData_1013() {
107        int baris_1013 = tabelMahasiswa_1013.getSelectedRow();
108
109        if (baris_1013 == -1) {
110            JOptionPane.showMessageDialog(this, "Pilih data yang ingin dihapus!");
111            return;
112        }
113
114        dataMahasiswa_1013.remove(baris_1013);
115        tampilkanData_1013();
116    }
117
118    // method untuk menampilkan data ke tabel
119    private void tampilkanData_1013() {
120        modelTabel_1013.setRowCount(0);
121
122        for (Mahasiswa_2511531013 mhs_1013 : dataMahasiswa_1013) {
123            modelTabel_1013.addRow(new Object[]{
124                mhs_1013.getNama_1013(),
125                mhs_1013.getNim_1013(),
126                mhs_1013.getProdi_1013()
127            });
128        }
129    }

```

```

130
131 // method untuk menjalankan sorting sesuai pilihan combo box
132 private void mulaiSorting_1013() {
133     if (dataMahasiswa_1013.isEmpty()) {
134         JOptionPane.showMessageDialog(this, "Data mahasiswa masih kosong!");
135         return;
136     }
137
138     prosesArea_1013.setText("");
139
140     String pilihan_1013 = comboSorting_1013.getSelectedItem().toString();
141
142     if (pilihan_1013.equals("Insertion Sort")) {
143         insertionSort_1013();
144     } else if (pilihan_1013.equals("Selection Sort")) {
145         selectionSort_1013();
146     } else {
147         bubbleSort_1013();
148     }
149
150     tampilkanData_1013();
151     prosesArea_1013.append("\nHasil akhir: " + namaToString_1013() + "\n");
152 }
153
154 // algoritma insertion sort berdasarkan nama mahasiswa
155 private void insertionSort_1013() {
156     prosesArea_1013.append("=== INSERTION SORT ===\n");
157
158     for (int i_1013 = 1; i_1013 < dataMahasiswa_1013.size(); i_1013++) {
159         Mahasiswa_2511531013 key_1013 = dataMahasiswa_1013.get(i_1013);
160         int j_1013 = i_1013 - 1;
161
162         while (j_1013 >= 0 &&
163             dataMahasiswa_1013.get(j_1013).getNama_1013()
164                 .compareToIgnoreCase(key_1013.getNama_1013()) > 0) {
165             dataMahasiswa_1013.set(j_1013 + 1, dataMahasiswa_1013.get(j_1013));
166             j_1013--;
167         }
168
169         dataMahasiswa_1013.set(j_1013 + 1, key_1013);
170
171         prosesArea_1013.append("Langkah " + i_1013 + ": "
172             + namaToString_1013() + "\n");
173     }
174 }
175
176 // algoritma selection sort berdasarkan nama mahasiswa
177 private void selectionSort_1013() {
178     prosesArea_1013.append("=== SELECTION SORT ===\n");
179
180     for (int i_1013 = 0; i_1013 < dataMahasiswa_1013.size() - 1; i_1013++) {
181         int minIndex_1013 = i_1013;
182
183         for (int j_1013 = i_1013 + 1; j_1013 < dataMahasiswa_1013.size(); j_1013++) {
184             if (dataMahasiswa_1013.get(j_1013).getNama_1013()
185                 .compareToIgnoreCase(dataMahasiswa_1013.get(minIndex_1013).getNama_1013()) < 0) {
186                 minIndex_1013 = j_1013;
187             }
188         }
189
190         Mahasiswa_2511531013 temp_1013 = dataMahasiswa_1013.get(i_1013);
191         dataMahasiswa_1013.set(i_1013, dataMahasiswa_1013.get(minIndex_1013));
192         dataMahasiswa_1013.set(minIndex_1013, temp_1013);
193
194         prosesArea_1013.append("Pass " + (i_1013 + 1) + ": "
195             + namaToString_1013() + "\n");
196     }
197 }
198
199 // algoritma bubble sort berdasarkan nama mahasiswa
200 private void bubbleSort_1013() {
201     prosesArea_1013.append("=== BUBBLE SORT ===\n");
202
203     for (int i_1013 = 0; i_1013 < dataMahasiswa_1013.size() - 1; i_1013++) {
204         for (int j_1013 = 0; j_1013 < dataMahasiswa_1013.size() - i_1013 - 1; j_1013++) {
205             if (dataMahasiswa_1013.get(j_1013).getNama_1013()
206                 .compareToIgnoreCase(dataMahasiswa_1013.get(j_1013 + 1).getNama_1013()) > 0) {
207                 Mahasiswa_2511531013 temp_1013 = dataMahasiswa_1013.get(j_1013);
208                 dataMahasiswa_1013.set(j_1013, dataMahasiswa_1013.get(j_1013 + 1));
209                 dataMahasiswa_1013.set(j_1013 + 1, temp_1013);
210             }
211         }
212
213         prosesArea_1013.append("Pass " + (i_1013 + 1) + ": "
214             + namaToString_1013() + "\n");
215     }
216 }

```

```

218 }
219
220 // method untuk menampilkan nama mahasiswa dalam bentuk array
221 private String namaToString_1013() {
222     String hasil_1013 = "[";
223
224     for (int i_1013 = 0; i_1013 < dataMahasiswa_1013.size(); i_1013++) {
225         hasil_1013 += dataMahasiswa_1013.get(i_1013).getNama_1013();
226
227         if (i_1013 < dataMahasiswa_1013.size() - 1) {
228             hasil_1013 += ", ";
229         }
230     }
231
232     hasil_1013 += "]";
233     return hasil_1013;
234 }
235
236 public static void main(String[] args) {
237     SwingUtilities.invokeLater(() -> {
238         GUI_Utama_2511531013 gui_1013 =
239             new GUI_Utama_2511531013();
240
241         gui_1013.setVisible(true);
242     });
243 }
244 }

```

Program GUI_Utama_2511531013 digunakan untuk mengelola data mahasiswa sekaligus melakukan proses sorting menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort berbasis GUI (Graphical User Interface) menggunakan Java Swing.

Pada program ini terdapat `ArrayList<Mahasiswa_2511531013>` bernama `dataMahasiswa_1013` yang digunakan untuk menyimpan seluruh data mahasiswa yang diinputkan pengguna.

Program menggunakan beberapa komponen GUI seperti `JTextField`, `JButton`, `JComboBox`, `JTable`, dan `JTextArea`. `JTextField` digunakan untuk menginput nama, NIM, dan program studi mahasiswa. `JButton` digunakan untuk tombol tambah data, hapus data, dan mulai sorting. `JComboBox` digunakan untuk memilih algoritma sorting yang ingin digunakan. `JTable` digunakan untuk menampilkan data mahasiswa, sedangkan `JTextArea` digunakan untuk menampilkan langkah-langkah proses sorting.

Constructor `GUI_Utama_2511531013()` digunakan untuk mengatur tampilan GUI seperti ukuran frame, layout, panel input, tabel data mahasiswa, combo box algoritma sorting, dan area visualisasi proses

sorting. Pada constructor juga ditambahkan event listener untuk setiap tombol agar program dapat menjalankan aksi tertentu ketika tombol ditekan.

Method `tambahData_1013()` digunakan untuk menambahkan data mahasiswa ke dalam `ArrayList`. Program akan mengambil input dari `TextField`, kemudian membuat objek mahasiswa baru dan menyimpannya ke dalam list. Setelah data berhasil ditambahkan, tabel akan diperbarui menggunakan method `tampilkanData_1013()`.

Method `hapusData_1013()` digunakan untuk menghapus data mahasiswa berdasarkan baris yang dipilih pada tabel. Jika pengguna belum memilih data, maka program akan menampilkan pesan peringatan menggunakan `JOptionPane`.

Method `tampilkanData_1013()` digunakan untuk menampilkan seluruh data mahasiswa ke dalam `JTable`. Program akan menghapus isi tabel sebelumnya, kemudian menambahkan seluruh data dari `ArrayList` ke tabel.

Method `mulaiSorting_1013()` digunakan untuk menjalankan proses sorting sesuai algoritma yang dipilih pada `JComboBox`. Jika pengguna memilih `Insertion Sort`, maka program akan menjalankan method `insertionSort_1013()`. Jika memilih `Selection Sort`, maka program menjalankan `selectionSort_1013()`. Sedangkan jika memilih `Bubble Sort`, maka program menjalankan `bubbleSort_1013()`.

Method `insertionSort_1013()` digunakan untuk mengurutkan data mahasiswa berdasarkan nama menggunakan algoritma `Insertion Sort`. Proses perbandingan nama dilakukan menggunakan method `compareToIgnoreCase()` agar sorting tidak membedakan huruf besar dan kecil. Setiap langkah sorting akan ditampilkan pada `JTextArea`.

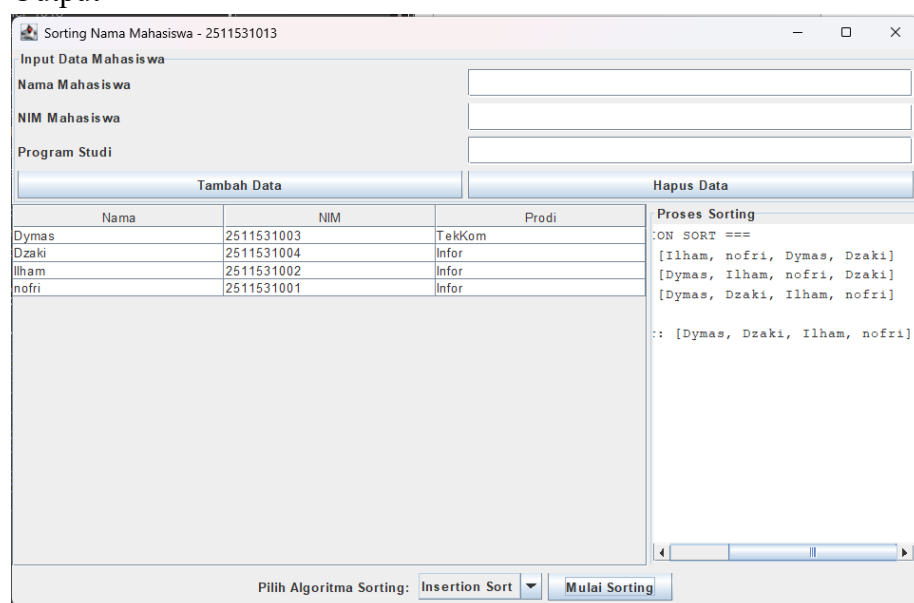
Method `selectionSort_1013()` digunakan untuk mengurutkan data menggunakan algoritma Selection Sort. Program mencari nama mahasiswa dengan urutan alfabet paling kecil, kemudian menukarnya dengan posisi data saat ini. Setiap proses sorting ditampilkan dalam bentuk pass pada area visualisasi.

Method `bubbleSort_1013()` digunakan untuk mengurutkan data menggunakan algoritma Bubble Sort. Program membandingkan dua data yang berdekatan, kemudian menukarnya jika urutannya salah. Proses ini dilakukan berulang sampai seluruh data terurut.

Method `namaToString_1013()` digunakan untuk mengubah isi nama mahasiswa pada `ArrayList` menjadi bentuk string agar lebih mudah ditampilkan pada area visualisasi sorting.

Pada method `main`, program dijalankan menggunakan `SwingUtilities.invokeLater()` agar GUI berjalan pada thread Swing. Setelah itu objek `GUI_Utama_2511531013` dibuat dan ditampilkan menggunakan `setVisible(true)`.

3. Output



Sorting Nama Mahasiswa - 2511531013

Input Data Mahasiswa

Nama Mahasiswa

NIM Mahasiswa

Program Studi

Tambah Data Hapus Data

Nama	NIM	Prodi
Dymas	2511531003	TekKom
Dzaki	2511531004	Infor
Ilham	2511531002	Infor
nofri	2511531001	Infor

Pilih Algoritma Sorting: Selection Sort Mulai Sorting

Proses Sorting

```
ACTION SORT ===  
[Dymas, Dzaki, Ilham, nofri]  
[Dymas, Dzaki, Ilham, nofri]  
[Dymas, Dzaki, Ilham, nofri]  
akhir: [Dymas, Dzaki, Ilham, nofri]
```

Sorting Nama Mahasiswa - 2511531013

Input Data Mahasiswa

Nama Mahasiswa

NIM Mahasiswa

Program Studi

Tambah Data Hapus Data

Nama	NIM	Prodi
Dymas	2511531003	TekKom
Dzaki	2511531004	Infor
Ilham	2511531002	Infor
nofri	2511531001	Infor

Pilih Algoritma Sorting: Bubble Sort Mulai Sorting

Proses Sorting

```
BUBBLE SORT ===  
[Dymas, Dzaki, Ilham, nofri]  
[Dymas, Dzaki, Ilham, nofri]  
[Dymas, Dzaki, Ilham, nofri]  
akhir: [Dymas, Dzaki, Ilham, nofri]
```